

Phase 2: Project Structure Generation

1. System Analysis

System Type: Web App and (Optional) Mobile App

Features:

- User authentication and authorization
- Appointment booking, updating, and canceling
- Doctor schedule management
- Patient record maintenance
- Report generation
- Email and SMS appointment reminders
- Admin user privileges

Non-Functional Requirements:

- Performance (2 seconds response time, supports 300 concurrent users)
- Security (encrypted data transmission and storage, role-based access)
- Usability (intuitive interfaces, clear error messages)
- Reliability (99% uptime, regular backups)
- Maintainability (easy user management, logging mechanism)
- Portability (supports common operating systems)

Complexity: Medium

2. Architecture

Style: Monolithic

Pattern: Model-View-Controller (MVC)

Justification:

Given the medium complexity of the system, a monolithic architecture provides a simpler and easier-to-manage solution for integrating multiple features such as user authentication, appointment management, doctor schedule management, patient record maintenance, report generation, email/SMS reminders, and admin privileges. The MVC design pattern can help structure the application's UI logic, business logic, and data management in a clean and maintainable way.

3. Tech Stack

Frontend: React.js (for web app) and React Native (for mobile app - optional)

Backend: Node.js with Express.js framework for API

Database: PostgreSQL or MongoDB (depending on the data model requirements)

Tools:

- JWT (JSON Web Tokens) for authentication and authorization
- Sequelize or Mongoose ORM (Object-Relational Mapper)

Justification:

React.js is a popular choice for building user interfaces due to its performance, component reusability, and robust ecosystem of libraries. For mobile apps, React Native offers cross-platform compatibility without compromising on functionality or user experience. Node.js and Express.js are

widely used for building scalable backends, especially in combination with JavaScript frontend technologies. PostgreSQL and MongoDB are both reliable databases that can handle the required data complexity and performance. JWT is a common practice for handling authentication in web applications, while Sequelize (for PostgreSQL) and Mongoose (for MongoDB) provide easy-to-use ORMs to interact with the respective databases.

4. Project Structure

```
...
frontend/
  node_modules/
  ...
  src/
    components/
    ...
    contexts/
    ...
    pages/
      Auth/
        Login.js
        Register.js
        ...
      Appointment/
        Booking.js
        Update.js
        Cancel.js
        ...
      DoctorSchedule/
        Schedule.js
        ...
      PatientRecord/
        Record.js
        ...
      Report/
        Generate.js
        ...
      Reminders/
        EmailReminder.js
        SMSReminder.js
        ...
      Admin/
        Dashboard.js
        ...
    services/
      authentication.js
      appointment.js
      doctorSchedule.js
      patientRecord.js
      report.js
      reminders.js
    utils/
    ...
  App.js
  index.js
  ...
```

```
■■■ package.json
■■■ package-lock.json
■■■ readme.md

backend/
■■■ node_modules/
■   ...
■■■ src/
■   ■■■ controllers/
■   ■   ■■■ authenticationController.js
■   ■   ■■■ appointmentController.js
■   ■   ■■■ doctorScheduleController.js
■   ■   ■■■ patientRecordController.js
■   ■   ■■■ reportController.js
■   ■   ■■■ remindersController.js
■   ■■■ models/
■   ■   ■■■ User.js
■   ■   ■■■ Appointment.js
■   ■   ■■■ DoctorSchedule.js
■   ■   ■■■ PatientRecord.js
■   ■   ■■■ Report.js
■   ■   ■■■ Reminder.js
■   ■■■ routes/
■   ■   ■■■ auth.js
■   ■   ■■■ appointment.js
■   ■   ■■■ doctorSchedule.js
■   ■   ■■■ patientRecord.js
■   ■   ■■■ report.js
■   ■   ■■■ reminders.js
■   ■■■ services/
■   ■   ...
■   ■■■ utils/
■   ■   ...
■   ■■■ app.js
■   ■■■ index.js
■   ■■■ ...
■■■ database.sql
■■■ package.json
■■■ package-lock.json
■■■ readme.md
^^^
```